

8 PySpark COD Solutions

Solutions prepared from the uploaded assessment PDF. The document contains one solution section for each of the 8 questions shown in the source.

Q19 – EASY 4: Media – Music Streaming Release Insights

Use the functions independently as required by the assessment. Imports are kept explicit and the code follows the operations specified in the question.

```
from pyspark.sql import DataFrame
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, to_date, year, month, date_trunc, sum

def load_music_tracks(spark: SparkSession, path: str) -> DataFrame:
    return (
        spark.read
        .option("header", True)
        .option("inferSchema", True)
        .csv(path)
        .withColumn("release_date", to_date(col("release_date")))
    )

def fill_missing_genre(df: DataFrame) -> DataFrame:
    return df.fillna({"genre": "Uncategorized"})

def add_release_calendar(df: DataFrame) -> DataFrame:
    return (
        df.withColumn("release_year", year(col("release_date")))
        .withColumn("release_month", month(col("release_date")))
        .withColumn("release_month_start", date_trunc("month", col("release_date")))
    )

def unique_published_artists(df: DataFrame) -> list[str]:
    return (
        df.filter(col("status") == "Published")
        .select("artist_name")
        .distinct()
        .orderBy("artist_name")
        .rdd.map(lambda r: r[0])
        .collect()
    )

def top_n_genres_by_streams(df: DataFrame, n: int) -> DataFrame:
    return (
        df.filter(col("status") == "Published")
        .groupBy("genre")
        .agg(sum("stream_count").alias("total_streams"))
        .orderBy(col("total_streams").desc(), col("genre").asc())
        .limit(n)
    )
```

Q24 – MEDIUM 1: BFSI – Merchant Settlement Risk Analytics

Use the functions independently as required by the assessment. Imports are kept explicit and the code follows the operations specified in the question.

```
from pyspark.sql import DataFrame
from pyspark.sql.types import StructType, StructField, StringType, DoubleType, IntegerType
from pyspark.sql.functions import col, coalesce, lit, concat_ws, sum, count
from pyspark.sql.window import Window
from pyspark.sql.functions import rank

def define_settlement_schema() -> StructType:
    return StructType([
        StructField("settlement_id", StringType(), True),
        StructField("merchant_id", StringType(), True),
        StructField("settlement_ts", StringType(), True),
        StructField("gross_amount", DoubleType(), True),
        StructField("fee_amount", DoubleType(), True),
        StructField("status", StringType(), True),
    ])

def load_settlement_data(spark, settlements_path: str, merchants_path: str, schema: StructType):
    settlements_df = (
        spark.read.option("header", True).schema(schema).csv(settlements_path)
        .withColumn("settlement_ts", col("settlement_ts").cast("timestamp"))
    )
    merchants_df = (
        spark.read.option("header", True).option("inferSchema", True).csv(merchants_path)
        .withColumn("risk_score", col("risk_score").cast("int"))
    )
    return settlements_df, merchants_df

def enrich_settlements(settlements_df: DataFrame, merchants_df: DataFrame) -> DataFrame:
    return (
        settlements_df.join(merchants_df, "merchant_id", "inner")
        .withColumn("city", coalesce(col("city"), lit("Unknown")))
        .withColumn("merchant_label", concat_ws(" - ", col("merchant_name"), col("city")))
        .withColumn("net_amount", col("gross_amount") - col("fee_amount"))
    )

def merchants_without_successful_settlement(merchants_df, settlements_df):
    successful = (
        settlements_df.filter(col("status") == "SUCCESS")
        .select("merchant_id").distinct()
    )
    return merchants_df.join(successful, "merchant_id", "left_anti")

def rank_merchants_by_net_amount(df: DataFrame) -> DataFrame:
    w = Window.orderBy(col("total_net_amount").desc())
    successful = df.filter(col("status") == "SUCCESS")
    return (
        successful.groupBy("merchant_id", "merchant_label")
        .agg(
            sum("net_amount").alias("total_net_amount"),
            count("settlement_id").alias("settlement_count")
        )
        .withColumn("settlement_rank", rank().over(w))
    )
```

Q26 – MEDIUM 3: Life Sciences – Bioreactor Run Performance Analytics

Use the functions independently as required by the assessment. Imports are kept explicit and the code follows the operations specified in the question.

```
from pyspark.sql import DataFrame
from pyspark.sql.types import StructType, StructField, StringType, DoubleType
from pyspark.sql.functions import col, unix_timestamp, when, coalesce, concat_ws
from pyspark.sql.window import Window
from pyspark.sql.functions import dense_rank

def define_run_schema() -> StructType:
    return StructType([
        StructField("run_id", StringType(), True),
        StructField("reactor_id", StringType(), True),
        StructField("process_type", StringType(), True),
        StructField("start_ts", StringType(), True),
        StructField("end_ts", StringType(), True),
        StructField("input_mass", DoubleType(), True),
        StructField("output_mass", DoubleType(), True),
        StructField("run_status", StringType(), True),
    ])

def load_bioreactor_data(spark, runs_path: str, reactors_path: str, schema: StructType):
    runs_df = (
        spark.read.option("header", True).schema(schema).csv(runs_path)
        .withColumn("start_ts", col("start_ts").cast("timestamp"))
        .withColumn("end_ts", col("end_ts").cast("timestamp"))
    )
    reactors_df = spark.read.option("header", True).option("inferSchema", True).csv(reactors_path)
    return runs_df, reactors_df

def compute_run_metrics(df: DataFrame) -> DataFrame:
    duration = (unix_timestamp(col("end_ts")) - unix_timestamp(col("start_ts"))) / 60
    return (
        df.withColumn("duration_minutes", when(duration < 0, 0).otherwise(duration))
        .withColumn(
            "yield_pct",
            when(col("input_mass") == 0, None)
            .otherwise((col("output_mass") / col("input_mass")) * 100)
        )
    )

def join_reactor_metadata(runs_df: DataFrame, reactors_df: DataFrame) -> DataFrame:
    return (
        runs_df.join(reactors_df, "reactor_id", "inner")
        .withColumn("facility", coalesce(col("facility"), lit("Unknown")))
        .withColumn("reactor_label", concat_ws(" - ", col("reactor_name"), col("facility")))
    )

def dense_rank_runs_by_yield(df: DataFrame) -> DataFrame:
    w = Window.partitionBy("process_type").orderBy(col("yield_pct").desc())
    return (
        df.filter((col("run_status") == "COMPLETED") & col("yield_pct").isNotNull())
        .withColumn("yield_rank", dense_rank().over(w))
        .select("run_id", "process_type", "yield_pct", "yield_rank")
    )
```

Q17 – EASY 2: Healthcare – Emergency Triage Wait-Time Insights

Use the functions independently as required by the assessment. Imports are kept explicit and the code follows the operations specified in the question.

```
from pyspark.sql import DataFrame, SparkSession
from pyspark.sql.functions import col, unix_timestamp, when, avg

def load_inline_triage_data(spark: SparkSession) -> DataFrame:
    rows = [
        ("C001", "ER", "2026-09-01 08:00:00", "2026-09-01 08:25:00", "High", "ACTIVE"),
        ("C002", "Trauma", "2026-09-01 08:10:00", "2026-09-01 08:05:00", "Critical", "ACTIVE"),
        ("C003", "Pediatrics", "2026-09-01 09:00:00", "2026-09-01 09:45:00", "Low", "ACTIVE"),
        ("C004", "ER", "2026-09-01 09:30:00", "2026-09-01 10:00:00", "High", "CLOSED"),
    ]
    columns = ["case_id", "department", "arrival_time", "doctor_start_time", "priority", "status"]
    return spark.createDataFrame(rows, columns)

def compute_wait_minutes(df: DataFrame) -> DataFrame:
    wait = (unix_timestamp(col("doctor_start_time")) -
            unix_timestamp(col("arrival_time"))) / 60
    return df.withColumn("wait_minutes", when(wait < 0, 0).otherwise(wait))

def filter_priority_cases(df: DataFrame) -> DataFrame:
    return df.filter(col("priority").isin("Critical", "High") & col("wait_minutes").isNotNull())

def average_wait_by_department(df: DataFrame) -> DataFrame:
    return df.groupBy("department").agg(avg("wait_minutes").alias("avg_wait_minutes"))

def list_active_departments(df: DataFrame) -> list[str]:
    return (
        df.filter(col("status") == "ACTIVE")
        .select("department").distinct()
        .orderBy("department")
        .rdd.map(lambda r: r[0]).collect()
    )
```

Q25 – MEDIUM 2: Healthcare – Remote Patient Monitoring Alert Analytics

Use the functions independently as required by the assessment. Imports are kept explicit and the code follows the operations specified in the question.

```
from pyspark.sql import DataFrame
from pyspark.sql.types import StructType, StructField, StringType
from pyspark.sql.functions import col, when, row_number, count, avg
from pyspark.sql.window import Window

def define_observation_schema() -> StructType:
    return StructType([
        StructField("observation_id", StringType(), True),
        StructField("patient_id", StringType(), True),
        StructField("reading_ts", StringType(), True),
        StructField("systolic", StringType(), True),
        StructField("heart_rate", StringType(), True),
        StructField("device_status", StringType(), True),
    ])

def load_monitoring_data(spark, observations_path: str, patients_path: str, schema: StructType):
    observations_df = (
        spark.read.option("header", True).schema(schema).csv(observations_path)
        .withColumn("reading_ts", col("reading_ts").cast("timestamp"))
        .withColumn("systolic", col("systolic").cast("int"))
        .withColumn("heart_rate", col("heart_rate").cast("int"))
    )
    patients_df = spark.read.option("header", True).option("inferSchema", True).csv(patients_path)
    return observations_df, patients_df

def classify_readings(df: DataFrame) -> DataFrame:
    return df.withColumn(
        "alert_level",
        when((col("systolic") >= 180) | (col("heart_rate") >= 130), "Critical")
        .when((col("systolic") >= 140) | (col("heart_rate") >= 100), "Warning")
        .otherwise("Normal")
    )

def latest_reading_per_patient(df: DataFrame) -> DataFrame:
    w = Window.partitionBy("patient_id").orderBy(col("reading_ts").desc())
    return (
        df.withColumn("row_num", row_number().over(w))
        .filter(col("row_num") == 1)
        .drop("row_num")
    )

def care_team_alert_summary(observations_df: DataFrame, patients_df: DataFrame) -> DataFrame:
    return (
        observations_df.join(patients_df, "patient_id", "inner")
        .filter(col("alert_level").isin("Critical", "Warning"))
        .groupBy("care_team")
        .agg(
            count("observation_id").alias("alert_count"),
            avg("heart_rate").alias("avg_heart_rate")
        )
    )
```

Q18 – EASY 3: Life Sciences – Vaccine Batch Stability Insights

Use the functions independently as required by the assessment. Imports are kept explicit and the code follows the operations specified in the question.

```
from pyspark.sql import DataFrame, SparkSession
from pyspark.sql.functions import col, to_date, date_add, datediff, current_date

def load_vaccine_batches(spark: SparkSession, path: str) -> DataFrame:
    return (
        spark.read.option("header", True).option("inferSchema", True).csv(path)
        .withColumn("manufacture_date", to_date(col("manufacture_date")))
        .withColumn("expiry_date", to_date(col("expiry_date")))
    )

def drop_incomplete_batches(df: DataFrame) -> DataFrame:
    return df.dropna(subset=[
        "batch_id", "product_name", "manufacture_date",
        "expiry_date", "potency_pct"
    ])

def add_stability_dates(df: DataFrame) -> DataFrame:
    return (
        df.withColumn("shelf_life_days",
            datediff(col("expiry_date"), col("manufacture_date")))
        .withColumn("review_date",
            date_add(col("manufacture_date"), 30))
        .withColumn("days_since_manufacture",
            datediff(current_date(), col("manufacture_date")))
    )

def filter_potency_range(df: DataFrame, low: float, high: float) -> DataFrame:
    return df.filter(col("potency_pct").between(low, high))

def count_release_ready_batches(df: DataFrame) -> int:
    return df.filter(
        (col("release_status") == "RELEASED") &
        col("potency_pct").isNotNull()
    ).count()
```

Q16 – EASY 1: BFSI – Digital Banking KYC Risk Insights

Use the functions independently as required by the assessment. Imports are kept explicit and the code follows the operations specified in the question.

```
from pyspark.sql import DataFrame
from pyspark.sql.functions import col, trim, when, avg, min, max, count

def load_kyc_data(spark, path: str) -> DataFrame:
    return (
        spark.read.option("header", True).option("inferSchema", True).csv(path)
        .withColumn("onboarding_date", col("onboarding_date").cast("date"))
    )

def remove_invalid_emails(df: DataFrame) -> DataFrame:
    cleaned = df.withColumn("email", trim(col("email")))
    return cleaned.filter(
        col("email").isNotNull() &
        (col("email") != "") &
        col("email").rlike(r"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$")
    )

def filter_review_customers(df: DataFrame, min_score: int, max_score: int) -> DataFrame:
    return df.filter(
        col("risk_score").between(min_score, max_score) &
        col("kyc_status").isin("PENDING", "REVIEW")
    )

def risk_score_statistics(df: DataFrame) -> dict:
    row = (
        df.filter(col("risk_score").isNotNull())
        .agg(
            min("risk_score").alias("min_score"),
            max("risk_score").alias("max_score"),
            avg("risk_score").alias("avg_score"),
            count("*").alias("total_customers")
        ).first()
    )
    return {
        "min_score": row["min_score"],
        "max_score": row["max_score"],
        "avg_score": row["avg_score"],
        "total_customers": row["total_customers"]
    }

def city_highest_average_risk(df: DataFrame):
    result = (
        df.filter(col("city").isNotNull() & col("risk_score").isNotNull())
        .groupBy("city")
        .agg(avg("risk_score").alias("average_risk"))
        .orderBy(col("average_risk").desc(), col("city").asc())
        .first()
    )
    return (result["city"], result["average_risk"]) if result else ("", 0.0)
```

Q27 – MEDIUM 4: Media – Streaming Ad Campaign Performance Analytics

Use the functions independently as required by the assessment. Imports are kept explicit and the code follows the operations specified in the question.

```
from pyspark.sql import DataFrame
from pyspark.sql.functions import col, when, sum, count

def load_ad_impressions(spark, path: str) -> DataFrame:
    return (
        spark.read.option("header", True).option("inferSchema", True).csv(path)
        .withColumn("impression_ts", col("impression_ts").cast("timestamp"))
        .withColumn("watched_seconds", col("watched_seconds").cast("double"))
        .withColumn("ad_length_seconds", col("ad_length_seconds").cast("double"))
        .withColumn("spend_amount", col("spend_amount").cast("double"))
    )

def join_campaign_metadata(impressions_df: DataFrame, campaigns_df: DataFrame) -> DataFrame:
    return impressions_df.join(campaigns_df, "campaign_id", "inner")

def add_engagement_metrics(df: DataFrame) -> DataFrame:
    watch_pct = when(
        col("ad_length_seconds").isNull() | (col("ad_length_seconds") <= 0),
        0.0
    ).otherwise(
        col("watched_seconds") / col("ad_length_seconds") * 100
    )

    return (
        df.withColumn("watch_pct", watch_pct)
        .withColumn(
            "engagement_band",
            when(col("watch_pct") >= 80, "High")
            .when(col("watch_pct") >= 40, "Medium")
            .otherwise("Low")
        )
    )

def campaign_performance_summary(df: DataFrame) -> DataFrame:
    return (
        df.filter(col("delivery_status") == "DELIVERED")
        .groupBy("campaign_id", "campaign_name")
        .agg(
            count("impression_id").alias("impression_count"),
            sum("spend_amount").alias("total_spend"),
            sum(when(col("clicked") == "Y", 1).otherwise(0)).alias("click_count")
        )
    )

def top_n_campaigns_by_spend(df: DataFrame, n: int) -> DataFrame:
    return (
        df.orderBy(col("total_spend").desc(), col("campaign_id").asc())
        .limit(n)
        .select(
            "campaign_id", "campaign_name", "total_spend",
            "impression_count", "click_count"
        )
    )
```